Software Design Principles

Theory Notes - April 2026

> *Software design principles are guidelines that help developers create systems that are easy to understand, maintain, and extend. They can be applied at both the high-level and low-level design stages.*

Three cornerstone principles:

- *DRY* - Don't Repeat Yourself
- *KISS* - Keep It Simple, Stupid
- *YAGNI* - You Aren't Gonna Need It

---

1 DRY: Don't Repeat Yourself

Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.

In simple terms → avoid duplication of logic or code. Repeating logic makes the system hard to maintain and error-prone.

↳ *If a change is required, you might forget to update all occurrences!*

Importance

- ✓ Reduces redundancy
- ✓ Easier maintenance
- ✓ Single point of change

Example - Bad Code (Violates DRY)

✗ Bad - Duplicated Logic

```
void printArea(int a, b) {
  int area = a * a;
  cout << "Total = " << area << endl;
}
void printArea(int a, b) {
  int area = a * a; // same logic repeated!
  cout << "Total = " << area << endl;
}
```

The logic for calculating area is repeated in both methods. Need to change the formula? → Must update in multiple places!

Refactored Code - Following DRY

✓ Good - DRY Principle Applied

```
class AreaCalculator {
  int calculateArea(length, width) {
    return length * width; // single source!
  }
}

// In main()
area1 = AreaCalculator.calculateArea(10, 5);
area2 = AreaCalculator.calculateArea(3, 4);
```

Now if we need to change the formula, we only need to do it in ONE place. ✓

Applying DRY in Practice

- → Identify repetitive code → replace with a single, reusable code segment.
- → Extract common functionality into utility classes.
- → Leverage libraries and frameworks when available.
- → Refactor duplicate logic regularly across classes or layers.

When NOT to Use DRY *Important!*

⚠ Caution

- *Premature Abstraction* - Don't extract common code too early. Two code blocks might look similar but could change differently later. Extracting them into a shared method can create unnecessary coupling between unrelated parts.
- *Performance-Critical Code* - Don't apply DRY to performance-sensitive code if it causes inefficiency. Sometimes, repeating optimized low-level logic is faster than calling a generalized, reusable method. Function calls, indirection, or generic wrapper might reduce performance or block compiler optimizations like inlining.
- *Sacrificing Readability* - If repeating repeated logic makes the code less readable, prefer clarity over DRY-ness.
- *Legacy Codebases* - Don't refactor for DRY's sake in legacy code unless necessary and well-tested. Introducing DRY by extracting shared logic can accidentally change behaviour.
- *Legacy code safety* - Often follow the "leave it alone unless you must touch it" rule.

> 💡 *Key Rule: DRY doesn't mean "never write similar-looking code" - it means don't duplicate knowledge or business logic without reason.*

2 KISS: Keep It Simple, Stupid

Complexity should be a key goal in design and unnecessary complexity should be avoided. Use the simplest possible solution that works. Avoid clever, convoluted code.

Importance

- ✓ Easier debugging
- ✓ Improved readability
- ✓ Better maintainability
- ✓ Faster development

Example

Writing a piece of code to check if a number is even or odd:

✗ Bad Code - Too Complex

```
class NumberChecker {
  static bool isEven(number) {
    if (number < 0) { number = -number; }
    if (number % 2 == 0) {
      isEven = true;
    } else {
      isEven = false;
    }
    return isEven;
  }
}
```

The above code is bad because it:

- ✗ Uses extra variables
- ✗ Adds unnecessary if-else logic
- ✗ Makes the code longer and harder to follow

✓ Good Code - Simple & Clear

```
class NumberChecker {
  static bool isEven(number) {
    return number % 2 == 0;
  }
}
```

The above code is good because it is:

- ✓ Simple, one-line solution
- ✓ Easy to read and understand
- ✓ Follows the DRY principle by avoiding over-engineering

> 🔑 *Remember: "Make it work, make it right, make it fast" - but keep it simple at every step. Complex code is a liability, not an asset!*

3 YAGNI: You Aren't Gonna Need It

"Always implement things when you actually need them, never when you just foresee that you need them."

In simple terms → don't add functionality until it's necessary. Avoid building things you think you might need. Helps keep the codebase clean and removes unnecessary complexity.

Example Scenario

Scenario

Suppose you've been asked to build a note-taking app that allows users to create and edit notes. You start thinking about: "What if they want categories? Or tagging? Or syncing with Google Drive?"

↳ *This creates a lot of unnecessary complexity before it's even needed!*

Importance

- ✓ Reduced complexity
- ✓ Simplified codebase
- ✓ Faster development

When NOT to Use YAGNI *Important!*

- → *When the requirements are well-known* - If a feature is guaranteed and easy to implement, preparing for it now might be more efficient.
  - ↳ e.g., You're building a messaging service that currently supports only text, but your product team has committed to supporting attachments in Q1. Handling attachments now might make sense here.
- → *Performance-Critical Areas* - In systems where performance is a first-class concern, resisting YAGNI might actually help. Proactively profiling and testing real-world usage patterns can catch bottlenecks early.

> ⚠ *Don't over-YAGNI: When requirements are known and clear, it's okay to build ahead. YAGNI is about avoiding speculative features, not refusing to plan!*

🔵 *DRY says...*
Don't duplicate knowledge or logic. Single source of truth.

🟠 *KISS says...*
Keep it simple. Avoid unnecessary complexity. Clarity is silverware.

🟢 *YAGNI says...*
Don't build what you don't need yet. Implement only when necessary.

💬 *Together these three principles help create codebases that are: clean, maintainable, scalable, and easy to reason about. Master them → write better software! ✓*